# Assignment 2

## Q1

Create a class `Car` with `brand` (string), `year` (int). Read N cars, store them, then print all.

**Sample Input:**

```
3
Toyota 2020
BMW 2022
Honda 2019
```

**Desired Output:**

```
Car 1: Toyota (2020)
Car 2: BMW (2022)
Car 3: Honda (2019)
```

---

## Q2

Create a class `Account` with private `balance`. Implement `deposit(amount)` (reject ≤ 0) and `withdraw(amount)` (reject if insufficient). Read operations and process.

**Sample Input:**

```
1000
4
deposit 500
withdraw 200
deposit -50
withdraw 9999
```

**Desired Output:**

```
Deposited 500 -> Balance: 1500

Withdrew 200 -> Balance: 1300

Error: invalid deposit

Error: insufficient funds

Final Balance: 1300
```

## Q3

Create a class `Tracer` that prints `"[ctor] <name>"` on construction and `"[dtor] <name>"` on destruction. Create 3 objects in a nested scope.

**Sample Input:**

```
Alpha
Beta
Gamma
```

**Desired Output:**

```
[ctor] Alpha
[ctor] Beta
[ctor] Gamma
[dtor] Gamma
[dtor] Beta
[dtor] Alpha
```

---

## Q4 — Copy Constructor & Deep Copy

Create a class `IntArray` wrapping a dynamic `int[]`. Implement a deep copy constructor. Read N ints, copy the array, modify the copy, print both.

**Sample Input:**

```
4
10 20 30 40
```

**Desired Output:**

```
Original: 10 20 30 40
Copy: 10 20 30 40
After copy[0]=99:
Original: 10 20 30 40
Copy: 99 20 30 40
```

---

## Q5 — Static Member: ID Generator

Create a class `User` with a `static int nextId`. Each user gets a unique auto-incremented ID. Read N names, create users, print them.

**Sample Input:**

```
3
Ali Sara Omar
```

**Desired Output:**

```
User #1: Ali
User #2: Sara
User #3: Omar
Total users: 3
```

---

## Q6 — Operator Overloading: Complex Numbers

Create a class `Complex` with `real` and `imag`. Overload `+`, `-`, and `<<`. Read two complex numbers, print sum and difference.

**Sample Input:**

```
3 4
1 2
```

**Desired Output:**

```
A = 3+4i
B = 1+2i
A+B = 4+6i
A-B = 2+2i
```

---

## Q7 — Operator Overloading: Comparison

Create a class `Student` with `name` and `gpa`. Overload `<` to sort by GPA descending. Read N students, sort, print ranking.

**Sample Input:**

```
4
Ali 3.5
Sara 3.9
Omar 3.2
Mona 3.7
```

**Desired Output:**

```
Rank 1: Sara (3.9)
Rank 2: Mona (3.7)
```

```
Rank 3: Ali (3.5)
Rank 4: Omar (3.2)
```

---

## Q8 — Friend Function

Create a class `Box` with private `length, width, height`. Write a friend function `volume(Box)`. Read dimensions and print volume.

**Sample Input:**

```
3 4 5
```

**Desired Output:**

```
Box(3x4x5) Volume: 60
```

---

## Q9 — Method Chaining (Fluent API)

Create `QueryBuilder` with `select(string)`, `from(string)`, `where(string)`, `build()` — each returns `*this`. Read table, columns, condition and build a query.

**Sample Input:**

```
users
name,age
age>18
```

**Desired Output:**

```
SELECT name,age FROM users WHERE age>18;
```

---

## Q10 — const Correctness

Create a class `Sensor` with private `double reading`. Implement `getReading() const` and `setReading(double)`. Show that a `const Sensor` can call `getReading()` but not `setReading()` (demonstrate in comments).

**Sample Input:**

```
25.7
```

**Desired Output:**

```
Sensor reading: 25.7
const sensor reading: 25.7
// constSensor.setReading(30) -> ERROR: const object
```

---

## Q11 — Subscript Operator

Create `SafeVector` wrapping `vector<int>`. Overload `[]` to print error for out-of-range. Read N ints and Q queries.

**Sample Input:**

```
5

10 20 30 40 50

4

0 2 4 7
```

**Desired Output:**

```
v[0] = 10

v[2] = 30

v[4] = 50

Error: index 7 out of range
```

---

## Q12 — Function Call Operator

Create a class `Adder` storing a base value. Overload `()` so `adder(x)` returns `base + x`. Read base and N queries.

**Sample Input:**

```
100

3

5 10 -20
```

**Desired Output:**

```
100 + 5 = 105

100 + 10 = 110

100 + -20 = 80
```

---

## Q13 — Explicit Constructor

Create a class `Meter` with `explicit` single-arg constructor. Show direct init works but implicit fails (in comments). Read a value.

**Sample Input:**

```
5.5
```

**Desired Output:**

```
Meter: 5.5m

// Meter m = 5.5; -> ERROR (explicit)

// func(5.5);     -> ERROR (explicit)
```

---

## Q14 — Mutable & const Method

Create `CallCounter` with `mutable int count`. Method `call(string msg) const` increments count. Read N messages.

**Sample Input:**

```
3

hello world test
```

**Desired Output:**

```
[1] hello

[2] world

[3] test

Total calls: 3
```

---

## Q15 — RAII Resource Management

Create `FileGuard` that prints "Opened " in ctor and "Closed " in dtor. Read a filename and use in a scoped block.

**Sample Input:**

```
data.csv
```

**Desired Output:**

```
Opened: data.csv

Processing file...

Closed: data.csv
```

---

## Q16 — Rule of Five Demonstration

Create a class `Buffer` managing `new char[]`. Implement all 5 special members. Print which one is called at each step.

**Sample Input:**

```
Hello
```

**Desired Output:**

```
Constructor: Hello

Copy ctor

Move ctor

Copy assign

Move assign

All destroyed
```

---

## Q17 — Composition (has-a)

Create `Engine(int hp)` and `Car(string name, Engine e)`. Read car data, print.

**Sample Input:**

```
Tesla 670

BMW 300
```

**Desired Output:**

```
Tesla: 670 HP

BMW: 300 HP
```

---

## Q18 — Aggregation: Course & Students

Create `Student(name)` and `Course(title)` where a course holds pointers to students. Read a course and its students.

**Sample Input:**

```
C++Programming

3

Ali Sara Omar
```

**Desired Output:**

```
Course: C++Programming

Students: Ali, Sara, Omar

Enrollment: 3
```

---

## Q19 — Struct vs Class

Create `struct Point{x,y}` (public) and `class Circle` with private `center` (Point) and `radius`. Read, print.

**Sample Input:**

```
3 4 5.0
```

**Desired Output:**

```
Point: (3, 4)

Circle: center=(3,4), radius=5, area=78.5398
```

---

## Q20 — Nested Class: Linked List

Create `LinkedList` with a nested `Node`. Implement `pushFront(int)` and `print()`. Read N ints.

**Sample Input:**

```
4

10 20 30 40
```

**Desired Output:**

```
40 -> 30 -> 20 -> 10 -> NULL
```

## Q21 — Basic Single Inheritance

Create `Person(name, age)` and `Student : Person` adding `gpa`. Read student data, print.

**Sample Input:**

```
Ali 20 3.8
```

**Desired Output:**

```
Name: Ali

Age: 20

GPA: 3.8
```

---

## Q22 — Multi-Level Inheritance

Create `LivingThing` → `Animal(name)` → `Dog(name, breed)`. Each adds a method. Read data, call all.

**Sample Input:**

```
Rex Labrador
```

**Desired Output:**

```
I am alive

I can move: Rex

I bark! I am a Labrador
```

---

## Q23 — Hierarchical Inheritance: Shapes

Create `Shape(name)` base. Derive `Circle(r)`, `Rectangle(w,h)`, `Triangle(b,h)`. Read shapes, print areas.

**Sample Input:**

```
3

circle 5

rectangle 4 6

triangle 3 8
```

**Desired Output:**

```
Circle: area=78.54

Rectangle: area=24.00

Triangle: area=12.00

Total: 114.54
```

---

## Q24 — Construction & Destruction Order

Create a 3-level hierarchy. Print traces. Read nothing — just observe order.

**Sample Input:** *(none)*

**Desired Output:**

```
Base ctor

Middle ctor

Derived ctor

---

Derived dtor

Middle dtor

Base dtor
```

---

## Q25 — Protected Members

Create `Vehicle` with `protected speed`. Derive `Car` with `accelerate(int)` and `brake(int)`. Read commands.

**Sample Input:**

```
4

acc 60

acc 40

brake 30

brake 100
```

**Desired Output:**

```
Speed: 60
```

```
Speed: 100

Speed: 70

Error: cannot brake by 100 (speed is 70)
```

---

## Q26 — Function Hiding vs Override

Create `Base` with non-virtual `info()` and virtual `describe()`. Override both in `Derived`. Call through `Base*`.

**Sample Input:** *(none)*

**Desired Output:**

```
Through Base*:

  info: Base (hiding!)

  describe: Derived (override!)

Direct Derived:

  info: Derived

  describe: Derived
```

---

## Q27 — Virtual Destructor

Create `Base` and `Derived` (prints in dtor). Delete through `Base*`. Show both dtors run.

**Sample Input:** *(none)*

**Desired Output:**

```
Creating Derived

Deleting via Base*:

~Derived

~Base
```

## Q28 — Multiple Inheritance

Create `Printable(print())` and `Saveable(save())`. Derive `Document` from both. Read a document name.

**Sample Input:**

```
report.pdf
```

**Desired Output:**

```
Printing: report.pdf

Saving: report.pdf
```

---

## Q29 — Diamond Problem

Create `Device(id)` → virtual `Sensor`, virtual `Actuator` → `SmartDevice`. Read an ID.

**Sample Input:**

```
42
```

**Desired Output:**

```
Device created: ID=42

SmartDevice ready

Device ID: 42 (single copy)
```

---

## Q30 — Abstract Base Class

Create abstract `Shape` with pure virtual `area()` and `perimeter()`. Implement `Circle` and `Rectangle`. Read shapes.

**Sample Input:**

```
2

circle 5

rect 3 4
```

**Desired Output:**

```
Circle: area=78.54 perimeter=31.42

Rectangle: area=12.00 perimeter=14.00
```

---

## Q31 — Interface: Multiple Implementation

Create interface `ILogger` with `log(string)`. Implement `ConsoleLogger` and `FileLogger`. Read log messages.

**Sample Input:**

```
3

Starting system

Loading config

Ready
```

**Desired Output:**

```
[Console] Starting system

[File] Starting system

[Console] Loading config

[File] Loading config
```

```
[Console] Ready

[File] Ready
```

---

## Q32 — override & final

Create `Base` → `Middle` (overrides and marks `final`) → `Leaf`. Show `override` catches errors.

**Sample Input:** *(none)*

**Desired Output:**

```
Middle::process (final)

// Leaf cannot override process

Leaf uses Middle::process
```

---

## Q33 — Inheritance vs Composition

Implement `Logger` class. Create `Service` using (a) inheritance and (b) composition. Both log a message.

**Sample Input:**

```
User created
```

**Desired Output:**

```
[Inherit] LOG: User created

[Compose] LOG: User created
```

---

## Q34 — Base Method Call from Derived

Create `Formatter::format(string)` that uppercases. `HTMLFormatter::format(string)` calls base then wraps in `<b>`.

**Sample Input:**

```
hello world
```

**Desired Output:**

```
Formatted: <b>HELLO WORLD</b>
```

---

## Q35 — Constructor Chaining

`A(int)` → `B(int,int)` → `C(int,int,int)`. Pass all up. Print from C.

**Sample Input:**

```
1 2 3
```

**Desired Output:**

```
A: x=1

B: y=2

C: z=3

Full: (1, 2, 3)
```

---

## Q36 — Private Inheritance

Create `Stack : private vector<int>`. Expose only `push`, `pop`, `top`, `size`. Read operations.

**Sample Input:**

```
6
```

```
push 10

push 20

push 30

top

pop

top
```

**Desired Output:**

```
Pushed 10

Pushed 20

Pushed 30

Top: 30

Popped

Top: 20
```

---

## Q37 — using Directive in Inheritance

Base has `print(int)` and `print(string)`. Derived overrides `print(int)` only. Use `using` to keep both visible.

**Sample Input:**

```
42

hello
```

**Desired Output:**

```
Derived::print(int): 42
```

```
Base::print(string): hello
```

---

## Q38 — Object Slicing

Create `Base{name}` and `Derived{name, extra}`. Show slicing by value vs safety by reference.

**Sample Input:** *(none)*

**Desired Output:**

```
By value: name=Test, extra=LOST

By reference: name=Test, extra=42
```

---

## Q39 — dynamic_cast Downcasting

Create `Animal` → `Dog(fetch)`, `Cat(purr)`. Store in `vector<Animal*>`, downcast to call specific methods.

**Sample Input:**

```
4

dog Rex

cat Mimi

dog Max

cat Luna
```

**Desired Output:**

```
Rex: Woof! *fetches ball*

Mimi: Meow! *purrs*

Max: Woof! *fetches ball*
```

```
Luna: Meow! *purrs*
```

---

## Q40 — Polymorphic Factory

Write `ShapeFactory::create(string)` returning `unique_ptr<Shape>`. Read shape commands.

**Sample Input:**

```
4
circle 5
rectangle 3 4
triangle 6 2
hexagon 3
```

**Desired Output:**

```
Circle: area=78.54
Rectangle: area=12.00
Triangle: area=6.00
Unknown shape: hexagon
```

## Q41 — Function Overloading

Write `area(double r)`, `area(double w, double h)`, `area(double b, double h, bool isTriangle)`. Read and compute.

**Sample Input:**

```
3
circle 5
rect 4 6
```

```
tri 3 8
```

**Desired Output:**

```
Circle: 78.54

Rectangle: 24.00

Triangle: 12.00
```

---

## Q42 — Template Function: Maximum

Write `template<typename T> T maximum(T a, T b)`. Test with int, double, char, string.

**Sample Input:**

```
10 20

3.14 2.71

a z

apple banana
```

**Desired Output:**

```
max(10,20) = 20

max(3.14,2.71) = 3.14

max(a,z) = z

max(apple,banana) = banana
```

---

## Q43 — Template Class: Pair

Create `template<typename T1, typename T2> class Pair` with `first`, `second`, and `print()`. Read pairs.

**Sample Input:**

```
Ali 95

3.14 2.71
```

**Desired Output:**

```
(Ali, 95)

(3.14, 2.71)
```

---

## Q44 — Template Stack

Create `template<typename T, int N> class FixedStack` with fixed capacity. Read push/pop operations.

**Sample Input:**

```
5

push 10

push 20

push 30

pop

top
```

**Desired Output:**

```
Pushed 10

Pushed 20

Pushed 30
```

```
Popped 30

Top: 20
```

---

## Q45 — Template Specialization

Write `template<typename T> string typeName()`. Specialize for `int`, `double`, `string`. Print type names.

**Sample Input:** *(hardcoded)*

**Desired Output:**

```
int -> integer

double -> floating-point

string -> text

bool -> unknown type
```

---

## Q46 — VTable Size Proof

Create `NoVirtual{int x}` and `WithVirtual{int x; virtual void f()}`. Print `sizeof` both.

**Sample Input:** *(none)*

**Desired Output:**

```
NoVirtual: 4 bytes

WithVirtual: 16 bytes (vptr + int + padding)
```

---

## Q47 — Strategy Pattern

Create `Sorter` accepting a `SortStrategy*`. Implement `AscSort` and `DescSort`. Read data, sort both ways.

**Sample Input:**

```
5

5 2 8 1 9
```

**Desired Output:**

```
Ascending: 1 2 5 8 9

Descending: 9 8 5 2 1
```

---

## Q48 — Observer Pattern

`Subject` notifies `Observer` list on value change. Read value changes.

**Sample Input:**

```
2

10

20
```

**Desired Output:**

```
Observer-A: value=10

Observer-B: value=10

Observer-A: value=20

Observer-B: value=20
```

---

## Q49 — NVI Pattern

`Document` has public `save()` calling private virtual `doSave()`. Derive `PDFDoc` and `WordDoc`.

**Sample Input:**

```
2

pdf

word
```

**Desired Output:**

```
Validating...

Saving PDF format

Done

Validating...

Saving Word format

Done
```

---

## Q50 — CRTP Instance Counter

Use CRTP `Counter<T>` to count instances per derived class. Create several `Widget` and `Button`.

**Sample Input:**

```
3 2
```

**Desired Output:**

```
Widgets created: 3

Buttons created: 2
```

---

## Q51 — Covariant Return

`Base::clone()` → `Base*`. `Derived::clone()` → `Derived*`. Clone and verify type.

**Sample Input:** *(none)*

**Desired Output:**

```
Original: Derived

Clone type: Derived (covariant)
```

---

## Q52 — Pure Virtual with Body

Abstract `Logger` has `log() = 0` with a body (timestamp). Derived `AppLogger` calls `Logger::log()` then adds its own message.

**Sample Input:**

```
System started
```

**Desired Output:**

```
[TIMESTAMP] System started

[APP] System started
```

---

## Q53 — Polymorphic Container Cleanup

Store shapes in `vector<unique_ptr<Shape>>`. Add shapes, print areas, let vector destroy them.

**Sample Input:**

```
3

circle 3

rect 2 5
```

```
tri 4 6
```

**Desired Output:**

```
Circle area: 28.27

Rect area: 10.00

Tri area: 12.00

~Triangle

~Rectangle

~Circle
```

---

## Q54 — typeid & RTTI

Create hierarchy. Print `typeid().name()` for objects through base pointers.

**Sample Input:** *(none)*

**Desired Output:**

```
Static type: Base

Dynamic types: Dog, Cat, Dog
```

---

## Q55 — Variadic Template Print

Write `template<typename... Args> void printAll(Args... args)` that prints each argument space-separated.

**Sample Input:** *(hardcoded)*

**Desired Output:**

```
42 3.14 hello true

1 2 3 4 5
```

## Q56 — auto & decltype

Read an int and double. Use `auto` for their sum. Print value and type (using typeid).

**Sample Input:**

```
10 3.14
```

**Desired Output:**

```
a=10 (int)

b=3.14 (double)

a+b=13.14 (double)
```

---

## Q57 — Range-based For: Transform

Read N ints. Use range-for to square each in place. Print before and after.

**Sample Input:**

```
5

1 2 3 4 5
```

**Desired Output:**

```
Original: 1 2 3 4 5

Squared: 1 4 9 16 25
```

---

## Q58 — Structured Bindings

Write `divide(int a, int b)` returning `tuple<int,int,bool>` (quotient, remainder, success). Use structured bindings.

**Sample Input:**

```
3

17 5

10 3

8 0
```

**Desired Output:**

```
17/5 = 3 remainder 2

10/3 = 3 remainder 1

8/0 = error: division by zero
```

---

## Q59 — unique_ptr

Create `unique_ptr<int[]>` of size N. Fill with squares. Transfer with `move`. Print from new owner.

**Sample Input:**

```
5
```

**Desired Output:**

```
Squares: 0 1 4 9 16

Owner transferred

Original is null: true
```

---

## Q60 — shared_ptr Counting

Create a `shared_ptr`, copy it twice inside a scope, print `use_count` at each step.

**Sample Input:** *(none)*

**Desired Output:**

```
Created: count=1

Copy 1: count=2

Copy 2: count=3

After scope: count=1

Destroyed
```

---

## Q61 — weak_ptr

Create `shared_ptr`, assign to `weak_ptr`, let shared expire, show `weak_ptr::expired()`.

**Sample Input:** *(none)*

**Desired Output:**

```
Alive: false (not expired)

Locked value: 42

After shared reset: expired=true
```

---

## Q62 — Move Semantics

Create `BigData(vector<int>(N))`. Copy vs move. Print sizes to prove move is O(1).

**Sample Input:**

```
1000000
```

**Desired Output:**

```
Original size: 1000000
```

```
After copy: orig=1000000, copy=1000000

After move: orig=0, moved=1000000
```

---

## Q63 — Lambda: Filter & Count

Read N ints and a threshold. Use a lambda with capture to filter and count elements above threshold.

**Sample Input:**

```
6 50

23 67 45 89 12 56
```

**Desired Output:**

```
Above 50: 67 89 56

Count: 3
```

---

## Q64 — Lambda: Custom Sort

Read N strings. Sort by length (ascending), then alphabetically for ties. Use lambda.

**Sample Input:**

```
5

banana fig apple kiwi cherry
```

**Desired Output:**

```
fig kiwi apple banana cherry
```

## Q65 — Generic Lambda (C++14)

Write a generic lambda `add` that works with int, double, and string. Test all three.

**Sample Input:**

```
3 4

1.5 2.5

Hello World
```

**Desired Output:**

```
3+4 = 7

1.5+2.5 = 4

Hello+World = HelloWorld
```

---

## Q66 — constexpr Factorial

Write `constexpr long long factorial(int)`. Use as array size (compile-time) and at runtime.

**Sample Input:**

```
6
```

**Desired Output:**

```
5! = 120 (compile-time array size)

6! = 720 (runtime)
```

---

## Q67 — std::optional

Write `optional<int> findFirst(vector<int>, predicate)`. Find first even number. Handle empty case.

**Sample Input:**

```
5

1 3 4 7 8

5

1 3 5 7 9
```

**Desired Output:**

```
First even: 4

No even number found
```

---

## Q68 — std::variant

Create `variant<int,double,string>` config map. Read key-type-value triples. Print with `visit`.

**Sample Input:**

```
3

port int 8080

rate double 3.14

host string localhost
```

**Desired Output:**

```
port = 8080 [int]

rate = 3.14 [double]
```

```
host = localhost [string]
```

---

## Q69 — enum class

Create `enum class Color{Red,Green,Blue}`. Write `toString(Color)` and `fromString(string)`. Read color names.

**Sample Input:**

```
3

Red Blue Green
```

**Desired Output:**

```
Red -> 0

Blue -> 2

Green -> 1
```

---

## Q70 — string_view

Write `int countChar(string_view sv, char c)`. Read a string and char. Also test with substring.

**Sample Input:**

```
Hello World

l
```

**Desired Output:**

```
"Hello World" has 3 'l's
```

```
Substr "World" has 1 'l'
```

---

## Q71 — if-with-init (C++17)

Read a `map<string,int>` of N entries. Then Q queries using `if (auto it = ...; ...)` syntax.

**Sample Input:**

```
3

Alice 95

Bob 87

Charlie 92

3

Alice Dave Bob
```

**Desired Output:**

```
Alice: 95

Dave: not found

Bob: 87
```

---

## Q72 — Fold Expression: Sum

Write variadic `sum(args...)` using fold. Print results from hardcoded calls.

**Sample Input:** *(none)*

**Desired Output:**

```
sum(1,2,3) = 6

sum(10,20,30,40) = 100
```

```
sum(1.5,2.5) = 4
```

---

## Q73 — if constexpr

Write `template<typename T> string classify(T)` using `if constexpr`. Read values.

**Sample Input:** *(hardcoded)*

**Desired Output:**

```
42 -> integer

3.14 -> floating-point

hello -> other type
```

---

## Q74 — Concepts (C++20): Numeric

Define `concept Numeric`. Write constrained `add()`. Show compile error for string (in comment).

**Sample Input:**

```
3 4

1.5 2.5
```

**Desired Output:**

```
3 + 4 = 7

1.5 + 2.5 = 4

// add("a","b") -> ERROR: not Numeric
```

---

## Q76 — vector: Dynamic Array

Read N ints into a `vector`. Print size, push_back one more, pop_back, print final.

**Sample Input:**

```
5

10 20 30 40 50

60
```

**Desired Output:**

```
Size: 5 -> 10 20 30 40 50

After push 60: 10 20 30 40 50 60

After pop: 10 20 30 40 50
```

---

## Q77 — vector: Erase & Insert

Read N ints. Erase element at index I, insert value V at index J. Print after each.

**Sample Input:**

```
5

10 20 30 40 50

2

1 99
```

**Desired Output:**

```
Original: 10 20 30 40 50

After erase[2]: 10 20 40 50

After insert 99 at [1]: 10 99 20 40 50
```

## Q78 — list: Doubly Linked List

Read N ints into a `list`. Insert at front and back. Remove first occurrence of X. Print.

**Sample Input:**

```
4

10 20 30 40

5

50

20
```

**Desired Output:**

```
Initial: 10 20 30 40

push_front 5: 5 10 20 30 40

push_back 50: 5 10 20 30 40 50

remove 20: 5 10 30 40 50
```

---

## Q79 — stack: Bracket Matching

Read a string of brackets. Use `stack<char>` to check if balanced.

**Sample Input:**

```
3

(()[]{})

((]

{[()]}
```

**Desired Output:**

```
(()[]{}) -> Balanced

(() -> Not Balanced

{[()]} -> Balanced
```

---

## Q80 — queue: Simulation

Simulate a printer queue. Read N jobs (name), process one per step (FIFO).

**Sample Input:**

```
4

report.pdf slides.pptx code.cpp image.png
```

**Desired Output:**

```
Queue: report.pdf slides.pptx code.cpp image.png

Processing: report.pdf

Processing: slides.pptx

Processing: code.cpp

Processing: image.png

Queue empty
```

---

## Q81 — priority_queue: Top K

Read N ints. Find top K largest using `priority_queue`.

**Sample Input:**

```
7 3

3 1 4 1 5 9 2
```

**Desired Output:**

```
Top 3: 9 5 4
```

---

## Q82 — map: Word Frequency

Read a line of words. Count frequency using `map<string,int>`. Print sorted alphabetically.

**Sample Input:**

```
the cat sat on the mat the cat
```

**Desired Output:**

```
cat: 2

mat: 1

on: 1

sat: 1

the: 3
```

---

## Q83 — unordered_map: Two Sum

Given N ints and a target, find two indices that sum to target using `unordered_map`.

**Sample Input:**

```
4 9
```

```
2 7 11 15
```

**Desired Output:**

```
Indices: 0 1 (2+7=9)
```

---

## Q84 — set: Unique Elements

Read N ints (with duplicates). Insert into `set`. Print unique sorted elements and count.

**Sample Input:**

```
8

4 2 7 2 9 4 1 7
```

**Desired Output:**

```
Unique: 1 2 4 7 9

Count: 5 (from 8 total)
```

---

## Q85 — multiset: Frequency

Read N ints into `multiset`. For each query value, print its count.

**Sample Input:**

```
8

1 2 2 3 3 3 4 4

3

2 3 5
```

**Desired Output:**

```
Count of 2: 2

Count of 3: 3

Count of 5: 0
```

---

## Q86 — deque: Sliding Window Max

Read N ints and window size K. Print the maximum in each window using `deque`.

**Sample Input:**

```
8 3

1 3 -1 -3 5 3 6 7
```

**Desired Output:**

```
3 3 5 5 6 7
```

---

## Q87 — algorithm: sort & reverse

Read N ints. Sort ascending, print. Reverse, print. Partial sort (top 3).

**Sample Input:**

```
6

5 2 8 1 9 3
```

**Desired Output:**

```
Sorted: 1 2 3 5 8 9
```

```
Reversed: 9 8 5 3 2 1

Top 3: 9 8 5
```

---

## Q88 — algorithm: find, count, accumulate

Read N ints. Find first occurrence of X, count X, compute sum.

**Sample Input:**

```
7

3 1 4 1 5 1 3

1
```

**Desired Output:**

```
First 1 at index: 1

Count of 1: 3

Sum: 18
```

---

## Q89 — algorithm: transform

Read N ints. Use `transform` to create a new vector of squares. Print both.

**Sample Input:**

```
5

1 2 3 4 5
```

**Desired Output:**

```
Original: 1 2 3 4 5

Squares: 1 4 9 16 25
```

---

## Q90 — algorithm: remove_if & erase

Read N ints. Remove all even numbers using erase-remove idiom. Print result.

**Sample Input:**

```
6

1 2 3 4 5 6
```

**Desired Output:**

```
Before: 1 2 3 4 5 6

After removing evens: 1 3 5
```

---

## Q91 — algorithm: unique

Read N sorted ints (with duplicates). Use `unique` + `erase` to remove consecutive duplicates.

**Sample Input:**

```
8

1 1 2 2 3 3 3 4
```

**Desired Output:**

```
Before: 1 1 2 2 3 3 3 4
```

```
After unique: 1 2 3 4
```

---

## Q92 — algorithm: binary_search & lower_bound

Read N sorted ints. For Q queries, check existence and find insertion point.

**Sample Input:**

```
6

1 3 5 7 9 11

3

5 6 11
```

**Desired Output:**

```
5: found, position=2

6: not found, would insert at position=3

11: found, position=5
```

---

## Q93 — algorithm: next_permutation

Read N ints. Print all permutations using `next_permutation`.

**Sample Input:**

```
3

1 2 3
```

**Desired Output:**

```
1 2 3
```

```
1 3 2

2 1 3

2 3 1

3 1 2

3 2 1
```

---

## Q94 — algorithm: min_element, max_element, minmax_element

Read N ints. Find min, max, and both together. Print values and indices.

**Sample Input:**

```
5

30 10 50 20 40
```

**Desired Output:**

```
Min: 10 at index 1

Max: 50 at index 2
```

---

## Q95 — algorithm: for_each with Lambda

Read N ints. Use `for_each` with a lambda to compute sum, count positives, and find max simultaneously.

**Sample Input:**

```
6

-3 7 2 -1 5 0
```

**Desired Output:**

```
Sum: 10

Positives: 3

Max: 7
```

---

## Q96 — algorithm: partition

Read N ints. Partition into even (left) and odd (right) using `partition`. Print.

**Sample Input:**

```
6

1 2 3 4 5 6
```

**Desired Output:**

```
Partitioned: 6 2 4 3 5 1

Even: 6 2 4 | Odd: 3 5 1
```

---

## Q97 — iterator: Custom Range Print

Write `printRange(begin, end)` using iterators. Read N ints, print full, first half, and last 3.

**Sample Input:**

```
6

10 20 30 40 50 60
```

**Desired Output:**

```
Full: 10 20 30 40 50 60

First half: 10 20 30

Last 3: 40 50 60
```

---

## Q98 — STL Combo: Phonebook

Use `map<string, vector<string>>` for name → phones. Support `add`, `lookup`, `remove` commands.

**Sample Input:**

```
6

add Ali 0123456

add Ali 0111111

add Sara 0999999

lookup Ali

remove Ali 0123456

lookup Ali
```

**Desired Output:**

```
Added 0123456 for Ali

Added 0111111 for Ali

Added 0999999 for Sara

Ali: 0123456, 0111111

Removed 0123456 from Ali

Ali: 0111111
```